

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS

Linguagem e Técnicas de Programação 2

ALUNOS:

Davi Augusto Marçal Rodrigues

Matheus Augusto de Carvalho Silva

Samuel Matias Almeida Irineu

TRABALHO PRÁTICO: MEGA MAN X

Contagem, 2025

Sumário

Sumário	2
1) Introdução	3
2) Desenvolvimento	4
2.1) Implementação	4
2.1.1 Classes Principais	4
2.1.1.1 Classe Jogo	4
2.1.1.2. Classe Entidade	4
2.1.1.3. Classe Personagem	5
2.1.1.4. Interface Inimigo	5
2.1.1.5. Classe GerenciadorColisoes	5
2.1.1.6. Enum TipoInimigo	5
2.2) Estratégias de Desenvolvimento	5
2.3) Principais Dificuldades	6
3) Considerações Finais	8
4) Referências Bibliográficas	13
Anexos	14
ANEXO A - Repositório do Projeto no GitHub	14

1) Introdução

O trabalho tem como foco a criação de um jogo usando Padrão de Projeto Iterator, na linguagem de programação Java, juntamente com a biblioteca de recursos audiovisuais LibGDX. O projeto consiste no desenvolvimento de uma versão simplificada do jogo Mega Man X. Os objetivos incluem entender o uso do Iterator, além de aprender aspectos do *framework*¹ LibGDX para o desenvolvimento de aplicações multimídia.

Este relatório aborda o processo de desenvolvimento do projeto, explicando as estratégias, estruturas de dados e classes utilizadas no programa. Além disso, apresentará as principais dificuldades durante a execução e o resultado obtido.

¹ Frameworks são estruturas de código reutilizáveis que fornecem elementos para ajudar no desenvolvimento de aplicações.

2) Desenvolvimento

2.1) Implementação

O projeto criado é uma versão do jogo Mega Man X. Inicialmente a tela apresenta um jogo de plataforma com inimigos e um personagem principal. O jogador procura chegar até o fim da fase e matar o chefe final, tendo obstáculos no jogo que vão de inimigos há buracos no mapa. A marcação de vidas do jogador é baseada em uma quantidade de vida base que ao longo do mapa pode ser perdida ao ser atacado. Já os inimigos têm uma vida que se esvai com base nos ataques do Mega Man, que retiram porcentagem de suas vidas até que eles morram. O jogo acaba com apenas duas formas, a primeira sendo a morte do jogador e a segunda sendo a derrota do Chefão ao final da fase.

O jogo utilizado como referência foi o Mega Man X. A jogabilidade possui algumas diferenças em relação ao jogo. A implementação se deu através das seguintes classes principais:

2.1.1 Classes Principais

2.1.1.1 Classe Jogo

A classe `Jogo` é responsável por gerenciar a janela e os elementos do jogo. As principais funções exercidas pelas demais classes ocorrem nela, além disso, ela renderiza todos os personagens e ataques, gerenciando suas funções. Essa classe herda características da classe `Game` (definida na biblioteca `LibGDX`) que permite a manipulação de telas.

2.1.1.2. Classe Entidade

A classe `Entidade` é a classe base para as demais classes do jogo. Seus principais atributos são `Sprite`, `Textura` e `posição`. Além disso, ela possui dois métodos para animação que são utilizados em todas as filhas de `Entidade`.

2.1.1.3. Classe Personagem

A classe `Personagem` herda as características de `Entidade`, e também adiciona métodos de `mover()`, `atacar()` e `morrer()` que são sobrescritos em todos os demais personagens. É a classe que utiliza polimorfismo no jogo.

2.1.1.4. Interface Inimigo

A interface `Inimigo` é implementada por todos os personagens que não são o Mega Man. Ela é usada para colidir com o personagem principal e ele toma dano ao encostar.

2.1.1.5. Classe GerenciadorColisoes

A classe `GerenciadorColisoes` gerencia todas as colisões do jogo. Colisões entre ataques e plataformas, ataques e personagens, Mega Man e inimigos e personagens e plataformas.

2.1.1.6. Enum TipoInimigo

No enum `TipoInimigo` todos tipos de ataques são “instanciados” diretamente e cada Ataque recebe um `TipoInimigo`.

2.2) Estratégias de Desenvolvimento

Durante o desenvolvimento do projeto diferentes técnicas foram utilizadas. O código foi separado em classes. Essa separação facilitou o seu entendimento, além disso, permitiu que o gerenciamento dos elementos do jogo fosse feito de forma organizada e intuitiva.

Outra estratégia, foi nomear as classes, funções e variáveis com nomes significativos, o que facilitou a cooperação entre os participantes do projeto. Além disso, o uso de mecanismos de herança foi importante para reduzir repetições no código e enxugá-lo, bem como reutilizá-lo.

Algumas técnicas mais específicas foram a criação da interface `Inimigo` e a forma que os ataques são gerenciados. Todos os personagens que não são o Mega Man, são inimigos e as funções que ela possui são: `getRect()` que pega o corpo do inimigo, `getDano()`, `getAtaquesAtivos()` que retorna um *ArrayList*,

`setPosXmegaMan()` e `tomarDano()`. Os ataques são gerenciados por *ArrayLists*. A cada ataque de algum personagem é criado um novo objeto de Ataque com suas especificações. Toda vez que um objeto de ataque encosta em algum personagem ou plataforma, ele é removido do *array*.

Por fim, a utilização do GitHub, plataforma de hospedagem de código, facilitou a colaboração entre os membros da equipe de desenvolvimento, garantindo que as alterações fossem facilmente acessíveis e documentadas. Isso foi útil para garantir que todos trabalhassem no mesmo código fonte, evitando redundâncias e conflitos de códigos.

2.3) Principais Dificuldades

Durante o desenvolvimento do jogo, diversas dificuldades técnicas foram enfrentadas, exigindo soluções criativas e pesquisa complementar.

A aplicação do padrão de projeto proposto inicialmente apresentou dificuldades significativas. No início do desenvolvimento, houve uma compreensão equivocada sobre a real necessidade de implementar o padrão *Iterator*, uma vez que já estávamos utilizando estruturas como *ArrayList*, o que gerou a falsa impressão de que sua aplicação seria redundante.

Essa interpretação equivocada revelou uma falha na compreensão conceitual dos padrões de projeto. No entanto, essa dificuldade foi superada por meio de estudos complementares e, especialmente, com o auxílio do professor Alisson Rodrigo dos Santos, durante uma orientação realizada no dia 16/07. A partir disso, foi possível entender a importância e a correta forma de aplicar o padrão *Iterator*, o que possibilitou sua implementação adequada dentro da estrutura do jogo.

A criação do mapa, juntamente com a configuração de suas colisões, apresentou-se como um dos primeiros desafios. A dificuldade estava principalmente no uso da ferramenta Tiled, cuja interface e lógica de configuração demandam conhecimento prévio. Este problema foi mitigado a partir da análise de um vídeo tutorial disponível no YouTube (inserir vídeo), o qual auxiliou na compreensão do funcionamento do software e na correta aplicação das colisões no mapa. A solução encontrada foi recriar o mapa do zero, com as colisões corretamente identificadas.

A respeito disso, o principal problema foi a colisão entre os personagens e as plataformas. Diversas inconsistências foram observadas, como ausência de colisão ou comportamentos não convencionais durante a interação entre personagem e plataforma. Ao recriar o mapa e mudar a lógica de definir a posição dos personagens após a colisão, o problema foi superado.

A detecção de colisão entre o personagem principal e os inimigos também apresentou problemas. Idealmente, ao entrar em contato com um inimigo, o personagem principal deveria receber dano ao tocar somente uma vez e parar de tomar dano. No entanto, ele estava morrendo ao encostar. A situação foi contornada criando uma variável de tempo de invulnerabilidade que é atualizado através do delta time da própria LibGDX. Somente quando ela chega em 0, permite que o Mega Man tome dano.

Por fim, a criação dos ataques também revelou-se um problema. Ao aplicar o padrão de projeto, os ataques não estavam sendo devidamente gerenciados, deixando o programa muito lento. Então optamos por utilizar *ArrayList* para gerenciar os ataques ativos de cada personagem. Além disso, toda vez que eles atacam, em especial, o Mega Man, a posição dos ataques resetava. Para consertar isso, a cada ataque dos personagens, um objeto Ataque é criado e são removidos do *array* quando encostam em algum personagem ou plataforma.

3) Considerações Finais

O desenvolvimento deste projeto teve como objetivo principal aplicar os conceitos de programação orientada a objetos (POO) por meio da biblioteca LibGDX, utilizando como base o jogo Mega Man X. A proposta visava atender a uma série de requisitos funcionais previamente estabelecidos, que abrangeram desde a estruturação do código até aspectos visuais e sonoros. Embora nem todos os objetivos tenham sido plenamente alcançados, foi possível implementar uma versão funcional do jogo, ainda que com limitações técnicas em determinados pontos.

O controle do personagem principal foi implementado com sucesso. O personagem Mega Man pode ser controlado utilizando o teclado, permitindo movimentações básicas como andar, pular e atacar. Entretanto, dificuldades relacionadas à fluidez e à precisão dos movimentos foram observadas, principalmente no que diz respeito à resposta aos comandos de ataque e à movimentação durante saltos. Tais falhas se deram, em grande parte, pela complexidade da lógica envolvida na física do personagem, que exigiu soluções alternativas e, ainda assim, não atingiu um grau satisfatório de fidelidade ao jogo original.

A animação do personagem, por sua vez, foi um dos pontos mais bem executados do projeto. O uso de sprites permitiu a criação de animações visivelmente fluidas para o personagem principal, inimigos e elementos como ataques e movimentos do chefe final. As sequências de animação foram integradas de forma coerente com os eventos do jogo, promovendo uma experiência visual consistente com a proposta estética do título original.

A geração de inimigos e obstáculos foi parcialmente implementada. O sistema conta com inimigos que surgem no cenário e possuem comportamento básico de movimentação e colisão. No entanto, alguns inimigos são carregados em posições bugadas nas plataformas, alguns ficam meio que flutuando. Ainda assim, o ambiente é povoado com elementos adversários que contribuem para a dinâmica do jogo.

No que se refere às regras do jogo, foi possível incorporar alguns aspectos essenciais, como o sistema de vidas e estruturas do mundo. Esses componentes

funcionam em nível básico, permitindo ao jogador perder vidas ao receber dano. Contudo, ainda há erros lógicos que afetam o funcionamento contínuo dessas regras, como erros de colisões e falhas nas transições de estado do personagem.

A implementação do padrão de projeto Iterator foi realizada com sucesso. Para coleções de personagens e inimigos foram implementadas seguindo o padrão. Essa aplicação favoreceu a modularidade do código e contribuiu para a clareza e manutenção das interações entre múltiplos objetos do jogo, como os personagens.

Quanto ao uso de recursos multimídia, o projeto apresentou resultados satisfatórios. Foram utilizadas imagens para sprites dos personagens, elementos do cenário, menus e interface gráfica, além de sons para ataques, ambientação e interações. A trilha sonora e os efeitos sonoros foram integrados ao jogo de forma funcional, o que contribuiu para uma experiência imersiva e coerente com a proposta.

O principal requisito não atendido foi de progressão de fases. Apenas uma única fase foi desenvolvida, devido à complexidade envolvida na criação e estabilização da primeira fase. A implementação de múltiplas fases implicaria na duplicação e adaptação de lógicas já complexas, o que se mostrou inviável no tempo e recursos disponíveis para o projeto.

A estrutura do jogo foi desenvolvida inteiramente utilizando Programação Orientada a Objetos. Todas as classes foram organizadas segundo os princípios da modularidade e encapsulamento, sem a utilização de variáveis globais ou funções isoladas na classe principal. A arquitetura do código respeita a lógica de separação de responsabilidades, facilitando a manutenção e a escalabilidade do projeto.

O polimorfismo foi corretamente aplicado por meio do uso de herança e métodos sobrescritos com *override* nas classes que compartilhavam comportamentos comuns. Essa prática se mostrou útil especialmente nas entidades inimigas e nas animações, onde diferentes objetos assumem comportamentos específicos baseados em uma interface comum, garantindo reutilização de código e flexibilidade.

Por fim, o requisito de versionamento via *GitHub* foi devidamente cumprido. O repositório do projeto foi mantido atualizado por meio de *branches* específicas para cada funcionalidade ou correção, com commits regulares realizados pelos integrantes do

grupo. Isso permitiu um acompanhamento estruturado do desenvolvimento, favorecendo a colaboração e o controle de versões conforme exigido.

Dessa forma, conclui-se que o projeto, embora não tenha atingido todos os objetivos propostos em sua totalidade, conseguiu cumprir com grande parte dos requisitos funcionais. As limitações observadas, principalmente nos aspectos de lógica e estabilidade, não invalidam o aprendizado adquirido durante o processo. Pelo contrário, as dificuldades enfrentadas permitiram aprofundar o conhecimento técnico dos integrantes, especialmente em tópicos como padrões de projeto, POO, controle de fluxo do jogo e integração multimídia. O projeto se mostra, portanto, um exercício valioso de consolidação prática dos conceitos aprendidos ao longo da disciplina.

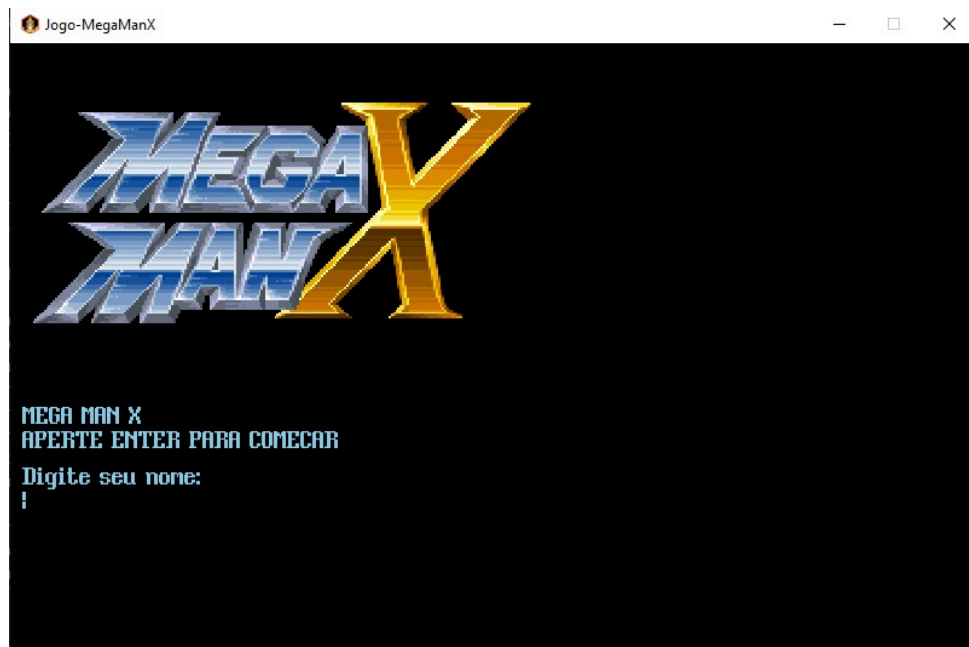


Figura 1: Representação da tela de início do jogo



Figura 2: Representação da tela de Game Over do jogo

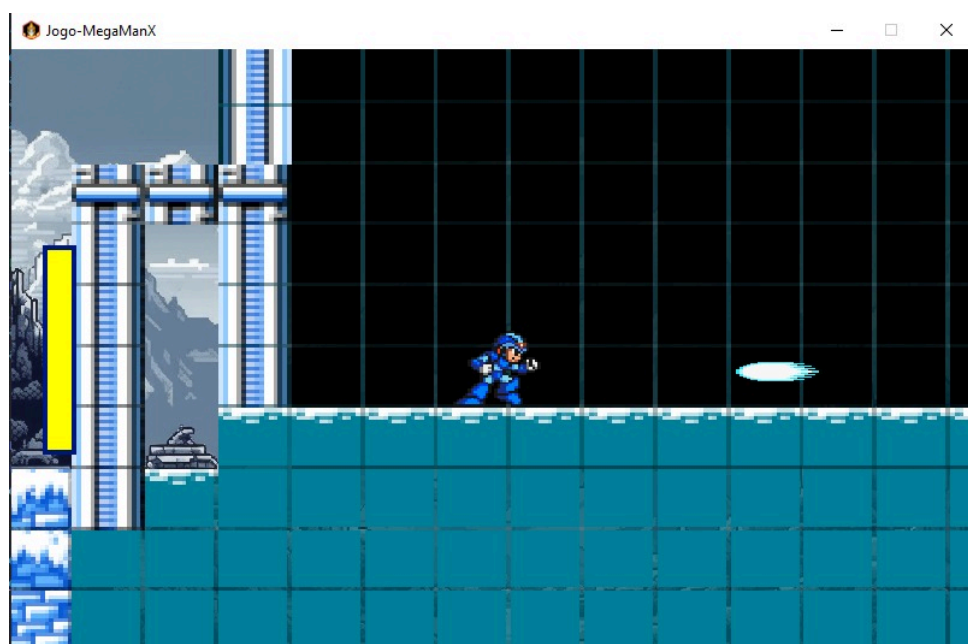


Figura 3: Representação do Mega Man atacando

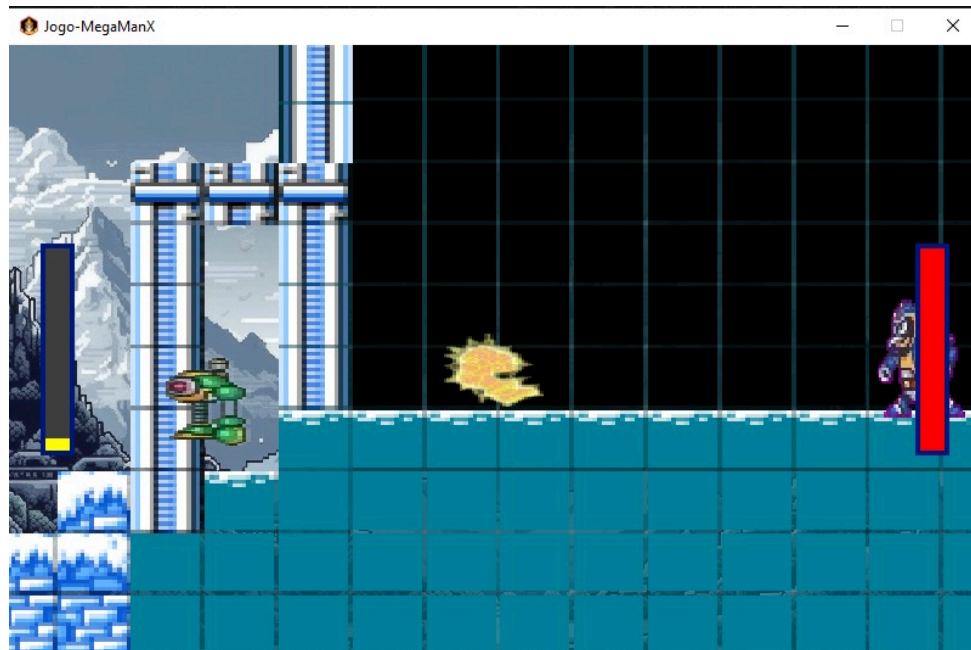


Figura 4: Representação do confronto entre Megaman e Chill Penguin

4) Referências Bibliográficas

CODE-DISASTER. MellowGame.java – Scrolling com LibGDX. GitHub, 2025. Disponível em: <https://github.com/code-disaster/gdx-mellow-demo/blob/master/core/src/com/codedisaster/mellow/MellowGame.java>. Acesso em: 5 jul. 2025.

REFACTORING GURU. Padrão de projeto Iterator. Disponível em: <https://refactoring.guru/pt-br/design-patterns/iterator>. Acesso em: 24 jun. 2025.

RETROGAMES.CZ. Mega Man X – Super Nintendo (SNES). [S.l.]: RetroGames.cz, [2025?]. Disponível em: https://www.retrogames.cz/play_865-SNES.php. Acesso em: 5 jul. 2025.

SAMUELMATIASSFR. DonkeyKongSFML [repositório GitHub], 2024. Disponível em: <https://github.com/SamuelMatiasSfr/DonkeyKongSFML>. Acesso em: 5 jul. 2025.

SPRITES INC. Mega Man X – Sprites. [S.l.]: Sprites Inc., [2025?]. Disponível em: <https://www.sprites-inc.co.uk/sprite.php?local=/X/>. Acesso em: 5 jul. 2025.

THE SPRITERS RESOURCE. Mega Man X (SNES) – Sprites. [S.l.]: The Spriters Resource. Disponível em: <https://www.sprisers-resource.com/snes/mmx/>. Acesso em: 5 jul. 2025.

YOUTUBE. Mecânica Scrolling. [S.l.]: YouTube, 2025. 1 vídeo. Disponível em: <https://www.youtube.com/watch?v=N-Sx-RcYvQc>. Acesso em: 5 jul. 2025.

YOUTUBE. Mega Man X – Gameplay. [S.l.]: YouTube, 2025. 1 vídeo. Disponível em: <https://www.youtube.com/watch?v=VLAASFngT3M>. Acesso em: 5 jul. 2025.

YOUTUBE. Padrões de Projeto em Java. [S.l.]: YouTube, 2025. 1 vídeo. Disponível em: https://www.youtube.com/watch?v=Xd_8hVDa61U. Acesso em: 27 jun. 2025.

YOUTUBE. Padrões de Projeto em Java – Série de vídeos. [S.l.]: YouTube, 2025. Vídeos 1, 2, 43 e 44. Disponível em: <https://www.youtube.com/watch?v=MqddY6Ochkc&list=PLbIBj8vQhvm0VY5YrMrafWaQY2EnJ3j8H>. Acesso em: 24 jun. 2025.

Anexos

ANEXO A - Repositório do Projeto no GitHub

O código-fonte completo do projeto está disponível no GitHub.

Link para o repositório: <https://github.com/Grupo-8-2025/TrabalhoPratico2>